

Documentation du déploiement — Vite & Gourmand

1. Choix de la plateforme d'hébergement

Plusieurs plateformes étaient envisageables : Fly.io, Heroku, Azure, Vercel.

Vercel a été écarté car il est plutôt optimisé pour les applications JavaScript/Next.js et il ne dispose pas d'un support natif pour PHP/Symfony.

Fly.io a également été écarté après recherche : le déploiement d'une application Symfony sur cette plateforme nécessite une configuration Docker avancée, représentant un risque technique important pour moi.

Azure aurait pu convenir (support PHP natif avec App Service) mais présente une interface de configuration plus complexe pour une première mise en production.

J'ai donc retenu Heroku, plateforme pour laquelle j'ai un accès gratuit pendant 24 mois grâce au GitHub Student Developer Pack. Le buildpack PHP officiel est bien documenté, le processus de déploiement est basé sur git push, et le tout était cohérent avec le flux Git déjà en place sur le projet.

2. Architecture de déploiement retenue

Heroku ne propose pas nativement de base de données MySQL (uniquement PostgreSQL en interne). L'application nécessitant donc, à la fois MySQL (Doctrine ORM) et MongoDB (Doctrine ODM, statistiques), une architecture distribuée a été mise en place :

L'application Symfony est hébergée sur Heroku. Elle se connecte à deux bases de données distinctes : JawsDB, un add-on Heroku qui fournit une base MySQL, et MongoDB Atlas, un service externe indépendant qui héberge la base NoSQL, connecté via la variable d'environnement `MONGODB_URI`.

Le stockage des images uploadées (VichUploaderBundle) a nécessité une étape supplémentaire, détaillée en section 6.

3. Étapes du déploiement

Préparation du code :

- Ajout d'un Procfile à la racine du projet, indiquant à Heroku comment démarrer l'application : `web: vendor/bin/heroku-php-apache2 public/`
- Ajout d'un fichier `public/.htaccess` (absent du squelette Symfony en environnement Apache local sous Windows, mais indispensable sur Heroku pour rediriger toutes les requêtes vers `index.php`). Son absence provoquait des erreurs 404 sur toutes les routes sauf la racine
- Configuration du buildpack : `heroku buildpacks:set heroku/php`

Création des ressources :

- `heroku create vite-et-gourmand-audrey`
- `heroku addons:create jawsdb:kitefin`

Un cluster MongoDB Atlas M0 a été créé indépendamment sur cloud.mongodb.com, avec :

- Un utilisateur de base de données dédié
- Une autorisation d'accès réseau 0.0.0.0/0 , rendue permanente (option "Make permanent"), nécessaire car Heroku ne dispose pas d'adresses IP statiques.

Configuration des variables d'environnement :

- heroku config:set DATABASE_URL=
«mysql://utilisateur:motdepasse@hote.amazonaws.com:3306/nom_base?serverVersion=8.0 »
- heroku config:set MONGODB_URI=
« mongodb+srv://utilisateur:motdepasse@cluster0.xxxxx.mongodb.net/nom_base?
retryWrites=true&w=majority »
- heroku config:set APP_ENV= prod
- heroku config:set APP_SECRET= « valeur générée aléatoirement »
- heroku config:set MAILER_DSN= « smtp://...@sandbox.smtp.mailtrap.io:2525 »

Déploiement du code :

- git push heroku develop:main

Heroku détecte automatiquement l'application PHP, installe les dépendances via Composer, exécute les scripts (cache:clear, assets:install , importmap:install), puis démarre l'application.

Initialisation de la base de données :

- heroku run "php bin/console doctrine:migrations:migrate --no-interaction"

Le bundle doctrine/doctrine-fixtures-bundle est installé en dépendance de développement (--dev). Composer l'exclut donc automatiquement en environnement de production (APP_ENV=prod). Les fixtures ne sont donc pas exécutables directement sur Heroku.

La base de données de production a été peuplée par export/import SQL direct (export de la base locale via MySQL Workbench, import dans JawsDB), après nettoyage des comptes de test personnels créés lors des phases de tests.

4. Vérifications après déploiement

- Test manuel effectué sur les différents parcours (catalogue, commande, connexion) sur l'URL de production
- Consultation des logs en temps réel pour diagnostiquer les erreurs 404 initiales (heroku logs -tail)
- Validation du score d'accessibilité (avec Lighthouse) directement sur l'environnement de production

5. Limitations identifiées et résolues

Persistance des fichiers uploadés :

Heroku utilise un système de fichiers éphémère : tout fichier écrit en dehors du dépôt Git (dossier public/uploads/, utilisé par VichUploaderBundle) est perdu à chaque redémarrage du

dyno. Ce redémarrage survient normalement après 30 minutes d'inactivité (mise en veille du plan Eco) ou lors de chaque nouveau déploiement. Ce problème, que j'ai tenté de contourner par une sauvegarde manuelle des images dans le dépôt Git, n'était finalement pas très fiable en pratique (l'image de démonstration avait disparu entre deux sessions de travail).

J'ai donc mis en place, comme solution, l'intégration de Cloudinary (service de stockage cloud d'images, avec plan gratuit permanent) comme backend de stockage pour VichUploaderBundle, avec league/flysystem-bundle et l'adaptateur thomasvantuycom/flysystem-cloudinary.

L'adaptateur Cloudinary n'étant pas un type natif du bundle Flysystem, sa déclaration a nécessité l'enregistrement manuel de trois services Symfony (client Cloudinary, adaptateur Flysystem, référence dans flysystem.yaml via le nom de classe de l'adaptateur).

L'option `use_flysystem_to_resolve_uri`: true a été activée dans la configuration de VichUploaderBundle. Elle est indispensable pour que le helper `vich_uploader_asset()` génère la véritable URL publique Cloudinary, plutôt qu'un chemin local incorrect.

Accès réseau MongoDB Atlas :

L'autorisation d'accès 0.0.0.0/0, était par défaut limitée à 7 jours maximum, avant expiration, lors de sa création. Elle a été configurée en accès permanent avec l'option dédiée du tableau de bord Atlas, pour garantir la disponibilité de la base sur la durée.

6. Bilan

Ce déploiement m'a permis de mettre en pratique une architecture multi-services (application + deux bases de données hébergées séparément), de diagnostiquer et de résoudre des problématiques concrètes de production (routage serveur, persistance de fichiers, réseau), qui dépassent le cadre d'un simple hébergement local.